Implemented Methods

SUQL and Trummer Heterogen Implementations

Code-grounded textbook notes for the Lab M2 repository. Includes method intuition, implementation paths, prompt examples, benchmark runners, usage commands, and bibliography.

Generated from docs/implemented_methods.md

July 6, 2026

Contents

This document explains the implemented SUQL and Trummer Heterogen methods in this repository. It is code-grounded: snippets are copied from the active implementation files, and each method section names the files that implement the behavior.

Chapter 1

High-Level Ideas

The repository studies query execution over a mixed IMDb workload: structured movie metadata plus unstructured movie reviews. The core question is how to answer natural-language predicates over reviews without sending unnecessary rows to a large language model. There are two main families of methods.

1. SUQL-style execution starts with structured predicates. A natural-language question is translated into a SQL-like query with `answer(review, ...)` and `summary(review)` functions. Normal SQL predicates run first in SQLite over a pandas DataFrame. Only rows that pass structured filters are sent to the LLM for review-text evaluation. This follows the SUQL paper's structured-first design [SUQL].

1. Trummer Heterogen-style execution treats the task as a semantic join between movie rows and review rows. The original block-join idea groups rows into prompts so a single LLM call can identify matching pairs [Trummer]. The repository then adds heterogeneous physical plans: deterministic ID joins, structured pruning, cheap-to-expensive cascades, and batched cascades.

The later implementations are inspired by the same systems problem as Stretto: choosing cheaper or more expensive physical operators under runtime-quality tradeoffs [Stretto]. They are not a full Stretto engine. They implement local operators for this benchmark: calibrated early exits, cheap binary scoring, fallback to expensive models, and batch-size choices.

Chapter 2

Bibliography

Citation key	Work	Used for	Repository connection
[SUQL]	Shicheng Liu, Jialiang Xu, Wesley Tjangnaka, Sina Semnani, Chen Yu, and Monica Lam. 2024. *SUQL: Conversational Search over Structured and Unstructured Data with Large Language Models*. Findings of NAACL 2024, pages 4535-4555. DOI: 10.18653/v1/2024.findings-naacl.283. https://aclanthology.org/2024.findings-naacl.283/	Structured-first query execution, `answer()`/`summary()`, semantic parsing into SQL-like queries.	`project SUQL/src_baseline/`, `project SUQL/src_baseline_stage1/`, `project SUQL/src_baseline_stage2/`.
[Trummer]	Immanuel Trummer. 2025. *Implementing Semantic Join Operators Efficiently*. arXiv:2510.08489. https://arxiv.org/abs/2510.08489	Tuple, block, and adaptive semantic join operators; prompt-level semantic joins.	`project Trummer/baseline/`, `project Trummer/heterogen_v1/`, `project Trummer/heterogen_v2_2/`.
[Stretto]	Gabriele Sanmartino, Matthias Urban, Paolo Papotti, and Carsten Binnig. 2026. *The Stretto Execution Engine for LLM-Augmented Data Systems*. arXiv:2602.04430. https://arxiv.org/abs/2602.04430	Runtime-quality tradeoffs, physical operator selection, cascades, and error-budget framing.	Stage 1, Stage 2, Heterogen V2/V2_3/V3/V3_2 cascades.
[ELEET]	Matthias Urban and Carsten Binnig. 2024. *Efficient Learned Query Execution over Text and Tables*. arXiv:2410.22522. https://arxiv.org/abs/2410.22522	Learned query execution over mixed text/table data and comparison to LLM-heavy execution.	Project positioning and comparison context.
[LLMxDATA]	Xuanhe Zhou et al. 2025. *A Survey of LLM x DATA*. arXiv:2505.18458. https://arxiv.org/abs/2505.18458	Broader LLM-for-data-management taxonomy and presentation context.	Background and literature framing.
[SUQL-code]	Stanford OVAL SUQL implementation. https://github.com/stanford-oval/suql	External implementation reference for SUQL concepts.	Used as design reference, not vendored as the active engine.
[LLMJoins-code]	Trummer semantic join code and IMDb review sample. https://github.com/itrummer/llmjoins	External semantic-join implementation reference.	`project Trummer/baseline/` and Heterogen prompt/operator design.

Citation key	Work	Used for	Repository connection
[IMDb50K]	IMDb 50K movie review dataset. https://www.kaggle.com/datasets/atulanandjha/imdb-50k-movie-reviews-test-your-bert	Movie review data provenance for Trummer baseline experiments.	Data source context for review workloads.

Chapter 3

Repository And Script Schema

```
.
|-- README.md
|-- docs/
|   `-- implemented_methods.md
|-- project SUQL/
|   |-- src_baseline/
|   |   |-- main.py
|   |   `-- suql_engine.py
|   |-- src_baseline_stage1/
|   |   |-- main.py
|   |   `-- suql_engine.py
|   |-- src_baseline_stage2/
|   |   |-- main.py
|   |   `-- suql_engine.py
|   |-- Stage_1/
|   |   |-- answer_filter.py
|   |   |-- profiler.py
|   |   |-- calibrate.py
|   |   |-- benchmark_stage1.py
|   |   `-- thresholds.json
|   |-- Stage_2/
|   |   |-- cascade_filter.py
|   |   |-- profiler.py
|   |   |-- calibrate.py
|   |   |-- benchmark_stage2.py
|   |   `-- thresholds.json
|   |-- data/
|   `-- data_samples/
|-- project Trummer/
|   |-- baseline/
|   |   |-- prepare_data.py
|   |   |-- run_experiment.py
|   |   `-- src/
|   |-- heterogen_v1/
|   |   |-- run_use_case3.py
|   |   `-- trummer_join/
|   |       |-- client.py
|   |       |-- data.py
|   |       `-- operators.py
|   |-- heterogen_v2/
|   |   |-- run_use_case3.py
|   |   `-- trummer_join/cascade.py
|   |-- heterogen_v2_2/
|   |   |-- run_use_case3.py
|   |   `-- trummer_join/
|   |       |-- operators.py
|   |       `-- structured_filter.py
|   |-- heterogen_v2_3/
|   |   |-- run_use_case3.py
|   |   `-- trummer_join/cascade.py
|   |-- heterogen_v3/
|   |   |-- run_use_case3.py
|   |   `-- trummer_join/
```

```
|   |       |-- cascade.py
|   |       `-- structured_filter.py
|   `-- heterogen_v3_2/
|       |-- run_use_case3.py
|       `-- trummer_join/
|           |-- cascade.py
|           `-- structured_filter.py
|-- common_benchmark/
|   `-- scripts/
|       |-- run_all.py
|       |-- run_suql_baseline.py
|       `-- run_trummer.py
|-- common_benchmark_v2/
|   `-- scripts/
|       |-- build_dataset.py
|       |-- run_all.py
|       |-- run_suql_baseline.py
|       `-- run_trummer.py
|-- common_benchmark_v3/
|   `-- scripts/
|       |-- run_all.py
|       |-- run_all_heterogen.py
|       |-- run_heterogen_v1.py
|       |-- run_heterogen_v2.py
|       |-- run_heterogen_v2_2.py
|       |-- run_heterogen_v2_3.py
|       |-- run_heterogen_v3.py
|       |-- run_suql_baseline.py
|       |-- evaluate_and_plot.py
|       `-- evaluate_all_heterogen.py
|-- common_benchmark_thresholds/
|   `-- scripts/run_threshold_sweep.py
|-- common_benchmark_3q/
|   `-- scripts/
|       |-- build_datasets.py
|       |-- run_all.py
|       |-- run_method.py
|       `-- evaluate_and_plot.py
`-- common_benchmark_10q/
    `-- scripts/
        |-- build_datasets.py
        |-- run_all.py
        |-- run_method.py
        `-- evaluate_and_plot.py
```

The common benchmark scripts are orchestration wrappers. The method implementations live under `project SUQL/` and `project Trummer/`; the benchmark directories load data, call those implementations, repeat runs, write metrics, and generate plots.

Chapter 4

Shared Benchmark Task Shape

Most one-question benchmark variants use the same movie-review task:

1. select movies with a structured condition such as `year = 1998`;
2. pair movies with reviews by `movie_id = tconst`;
3. decide whether the review expresses the requested semantic condition.

The implementations differ in where each condition is applied:

Method	**Structured movie condition**	**ID join**	**Review semantic predicate**
SUQL baseline	SQLite structured SQL	implicit in joined table	expensive `answer()` calls
Heterogen V1	inside block prompt	inside block prompt and parser validation	inside block prompt
Heterogen V2	not used	deterministic exact-ID candidate generation	cheap scorer plus expensive fallback
Heterogen V2_2	deterministic SUQL-style pruning	deterministic review pruning by selected IDs	expensive block prompts
Heterogen V2_3	not used	deterministic exact-ID candidate generation	batched cheap scorer plus batched fallback
Heterogen V3	deterministic SUQL-style pruning	deterministic review pruning by selected IDs	row-wise cheap scorer plus fallback
Heterogen V3_2	deterministic SUQL-style pruning	deterministic review pruning by selected IDs	batched cheap scorer plus batched fallback

Chapter 5

SUQL Baseline

5.1 Idea

The SUQL baseline follows a structured-first plan. The LLM first translates a natural-language question to a SQL-like query. The engine removes `answer()` predicates before running SQLite, so normal predicates such as `year`, `runtime`, `director`, and `genres` reduce the candidate set. Then it evaluates `answer(review, ...)` only on those candidates.

5.2 Code path

- `project SUQL/src_baseline/main.py`
- `project SUQL/src_baseline/suql_engine.py`
- Benchmark wrapper: `common_benchmark_v3/scripts/run_suql_baseline.py`
- Ten-question wrapper: `common_benchmark_10q/scripts/run_method.py`, function `run_suql()`

5.3 Semantic parser prompt

From `project SUQL/src_baseline/suql_engine.py`:

```
PARSER_SYSTEM = textwrap.dedent(f"""
You are a SUQL (Structured and Unstructured Query Language) semantic parser.
SUQL extends SQL with two free-text functions:
  - answer(free_text_column, 'question')  -> 'Yes' | 'No' | short string
  - summary(free_text_column)             -> a short prose summary

{TABLE_SCHEMA}

Rules:
1. Use plain SQL WHERE clauses for ALL structured predicates (year, runtime, director, genres,
     title, movie_id).
2. Use answer() ONLY for predicates that require understanding the review text.
3. Always apply structured filters BEFORE answer() in the WHERE clause (efficiency).
4. The SELECT list must include: movie_id, title, year, runtime, director, genres.
   Add summary(review) AS review_summary whenever you use answer() on the review.
5. Use LIKE '%value%' for partial text matches on genres and director.
6. For "top N" questions without a numeric ranking field, use LIMIT N.
7. Output ONLY the raw SQL query -- no markdown fences, no explanation.

Few-shot examples:
{FEW_SHOT_EXAMPLES}
""").strip()
```

Example few-shot entry from the same file:

```
-- Question: Top 5 drama movies from the 1990s that reviewers consider masterpieces?
```

```
SELECT movie_id, title, year, runtime, director, genres, summary(review) AS review_summary
FROM movies
WHERE genres LIKE '%Drama%'
  AND year >= 1990 AND year <= 1999
  AND answer(review, 'Does the reviewer consider this movie a masterpiece or give it very high
     praise?') = 'Yes'
ORDER BY year DESC
LIMIT 5;
```

5.4 Answer and summary operators

The baseline answer prompt is intentionally short and row-local:

```
ANSWER_SYSTEM = textwrap.dedent("""
You are evaluating a single movie review to answer a question about it.

Rules:
- If the question is a yes/no question, answer ONLY with 'Yes' or 'No' (no other text).
- If the question asks for a specific piece of information (e.g. a name, year, rating),
  answer with a brief string (a few words at most).
- Do not explain your reasoning. Output only the answer.
""").strip()
```

The row-level implementation truncates long reviews and caches repeated `(review, question)` calls:

```
def answer_fn(review_text: str, question: str) -> str:
    key = _cache_key(review_text, question)
    if key in _answer_cache:
        return _answer_cache[key]

    if not review_text or len(review_text.strip()) < 10:
        result = "No"
    else:
        prompt = f"Review:\n{review_text[:1500]}\n\nQuestion: {question}"
        result = _llm_call(ANSWER_SYSTEM, prompt, max_tokens=20)
        low = result.lower().strip().rstrip(".")
        if low in ("yes", "no"):
            result = low.capitalize()

    _answer_cache[key] = result
    return result
```

5.5 Execution details

The engine uses regexes to find `answer()` and `summary()` calls, removes unsupported function calls before SQLite execution, then applies semantic predicates after the structural query:

```
_ANSWER_RE = re.compile(
    r"answer\s*\(\s*(\w+)\s*,\s*(['\"])(.*?)\2\s*\)",
    re.IGNORECASE | re.DOTALL,
)

def _strip_answer_predicates(sql: str) -> str:
    cleaned = re.sub(
        r"\bAND\s+answer\s*\([^)]+\)\s*=\s*['\"][^'\"]*['\"]",
        "",
        sql,
        flags=re.IGNORECASE,
```

```
    )
    cleaned = re.sub(
        r"\bWHERE\s+answer\s*\([^)]+\)\s*=\s*['\"][^'\"]*['\"]",
        "WHERE 1=1",
        cleaned,
        flags=re.IGNORECASE,
    )
    return cleaned
```

5.6 Simple usage

```
cd "/Users/annremizova/Desktop/lab m2/project SUQL/src_baseline"
python main.py "Which horror movies under 110 minutes have reviews mentioning suspense or
    tension?"
```

Run through a shared benchmark:

```
cd "/Users/annremizova/Desktop/lab m2"
python3 common_benchmark_v3/scripts/run_suql_baseline.py \
  --api-base http://127.0.0.1:11434 \
  --model ollama/qwen3:1.7b \
  --output-dir /tmp/suql_baseline_example
```

Chapter 6

SUQL Stage 1: Calibrated Early Exit

6.1 Idea

Stage 1 keeps the SUQL query engine but replaces direct `answer()` calls with a calibrated filter. Every candidate is scored by a binary log-odds scorer. If the score is above the accept threshold, the row is accepted without full generation. If it is below the reject threshold, the row is rejected. Ambiguous rows fall back to the full answer prompt. This is a physical-operator change, not a new query language.

6.2 Code path

- `project SUQL/src_baseline_stage1/suql_engine.py`
- `project SUQL/Stage_1/answer_filter.py`
- `project SUQL/Stage_1/profiler.py`
- `project SUQL/Stage_1/thresholds.json`
- Benchmark script: `project SUQL/Stage_1/benchmark_stage1.py`

6.3 Scoring prompt

The binary scorer prompt comes from `project SUQL/Stage_1/profiler.py`:

```
@staticmethod
def _prompt(review: str, question: str) -> str:
    return (
        "Classify whether the review answers the question with Yes.\n"
        "Return exactly one token: 1 for Yes, 0 for No.\n\n"
        f"Question: {question}\n"
        f"Review: {review[:1800]}\n\n"
        "Answer:"
    )
```

6.4 Routing logic

From `project SUQL/Stage_1/answer_filter.py`:

```
threshold = self.thresholds.get(question)
if threshold is None:
    result = self.full_answer(review_text, question)
    self.cache[key] = result
    return result

self.stats.llm_score_calls += 1
try:
    log_odds = self.scorer.score(review_text, question)
```

```
except Exception:
    self.stats.llm_score_failures += 1
    result = self.full_answer(review_text, question)
    self.cache[key] = result
    return result

if log_odds >= threshold.accept_threshold:
    self.stats.llm_early_accept += 1
    self.cache[key] = "Yes"
    return "Yes"
if log_odds <= threshold.reject_threshold:
    self.stats.llm_early_reject += 1
    self.cache[key] = "No"
    return "No"

result = self.full_answer(review_text, question)
```

The fallback uses the same short answer system prompt as the baseline:

```
user_prompt = f"Review:\n{review_text[:1500]}\n\nQuestion: {question}"
native_payload = {
    "model": self.model.removeprefix("ollama/"),
    "messages": [
        {"role": "system", "content": ANSWER_SYSTEM},
        {"role": "user", "content": user_prompt},
    ],
    "stream": False,
    "options": {"temperature": 0, "num_predict": 20},
}
```

6.5 Simple usage

```
cd "/Users/annremizova/Desktop/lab m2/project SUQL"
python Stage_1/benchmark_stage1.py \
  --sample-size 100 \
  --model ollama/phi4-mini \
  --api-base http://127.0.0.1:11434
```

Chapter 7

SUQL Stage 2: Cheap-To-Expensive Cascade

7.1 Idea

Stage 2 separates the cheap scorer from the expensive answer model. The cheap model scores every candidate unless disabled or skipped. Confident positive scores become early accepts; confident negative scores become early rejects; uncertain candidates use the expensive full-answer model. It can learn a confidence threshold from a calibration sample, or use `SUQL_MANUAL_CONFIDENCE_THRESHOLD`.

7.2 Code path

- `project SUQL/src_baseline_stage2/suql_engine.py`
- `project SUQL/Stage_2/cascade_filter.py`
- `project SUQL/Stage_2/profiler.py`
- `project SUQL/Stage_2/thresholds.json`
- Benchmark script: `project SUQL/Stage_2/benchmark_stage2.py`
- Ten-question wrapper: `common_benchmark_10q/scripts/run_method.py`, function `run_suql_stage2()`

7.3 Model configuration

From `project SUQL/src_baseline_stage2/suql_engine.py`:

```
MODEL = _litellm_model_name(os.environ.get("SUQL_MODEL", "ollama/phi4-mini"))
CHEAP_MODEL = _litellm_model_name(os.environ.get("SUQL_CHEAP_MODEL", "ollama/gemma2:2b"))
EXPENSIVE_MODEL = _litellm_model_name(os.environ.get("SUQL_EXPENSIVE_MODEL", MODEL))
CHEAP_ACCEPT_FLOOR = float(os.environ.get("SUQL_CHEAP_ACCEPT_FLOOR", "4.0"))
CASCADE_TARGET = float(os.environ.get("SUQL_CASCADE_TARGET", "0.9"))
CALIBRATION_BUDGET = int(os.environ.get("SUQL_CALIBRATION_BUDGET", "20"))
```

7.4 Routing logic

From `project SUQL/Stage_2/cascade_filter.py`:

```
if self.manual_confidence_threshold is None:
    routing_threshold = self._learn_confidence_threshold(scored, question, usage)
    learned_threshold = routing_threshold
else:
    routing_threshold = self.manual_confidence_threshold
    learned_threshold = None
```

```
for index, review_text, score in scored:
    if _is_confident(score, routing_threshold) and score >= 0:
        self.stats.cheap_early_accept += 1
        self.cache[key] = "Yes"
        results[index] = "Yes"
    elif _is_confident(score, routing_threshold):
        self.stats.cheap_early_reject += 1
        self.cache[key] = "No"
        results[index] = "No"
    else:
        result = self.expensive_answer(review_text, question)
        self.cache[key] = result
        results[index] = result
```

Confidence is symmetric:

```
def _is_confident(score: float, threshold: float | None) -> bool:
    return threshold is not None and abs(score) >= threshold
```

7.5 Expensive prompt

```
user_prompt = f"Review:\n{review_text[:1500]}\n\nQuestion: {question}"
payload = {
    "model": self.expensive_ollama_model,
    "messages": [
        {"role": "system", "content": ANSWER_SYSTEM},
        {"role": "user", "content": user_prompt},
    ],
    "stream": False,
    "options": {"temperature": 0, "num_predict": 20},
}
```

7.6 Simple usage

```
cd "/Users/annremizova/Desktop/lab m2/project SUQL"
python Stage_2/benchmark_stage2.py \
  --sample-size 100 \
  --seed 11 \
  --api-base http://127.0.0.1:11434 \
  --model ollama/qwen3:1.7b \
  --cheap-model ollama/qwen3:0.6b
```

Chapter 8

Trummer Heterogen V1: Bounded Block Semantic Join

8.1 Idea

V1 is closest to the Trummer block-join idea. It partitions movies and reviews into blocks, creates one prompt per movie-block x review-block pair, and asks the LLM to return all matching movie IDs from the current block. The prompt contains only the current two blocks and the predicate. There is no conversation history and no whole-dataset context in the LLM call.

8.2 Code path

- `project Trummer/heterogen_v1/run_use_case3.py`
- `project Trummer/heterogen_v1/trummer_join/operators.py`
- `project Trummer/heterogen_v1/trummer_join/client.py`
- Benchmark wrapper: `common_benchmark_v3/scripts/run_heterogen_v1.py`

8.3 Block prompt

From `project Trummer/heterogen_v1/trummer_join/operators.py`:

```
def create_prompt(block_1: list[dict], block_2: list[dict], predicate: str) -> str:
    parts = [
        "Find every movie in Collection 1 that has a review in Collection 2 "
        "satisfying this predicate:",
        predicate,
        "Collection 1 contains structured movie rows. Collection 2 contains review chunks.",
        "A review belongs to a movie only when tconst is exactly equal to movie_id.",
        'Return JSON only: {"matching_movie_ids":["tt123"]}.',
        'If there are no matches, return {"matching_movie_ids":[]}.',
        "Copy each returned movie_id exactly from Collection 1.",
        "Do not add prose or a completion marker.",
        "",
        "Collection 1:",
    ]
    for idx, row in enumerate(block_1, 1):
        parts.append(f"{idx}: {row['text']}")
    parts.append("")
    parts.append("Collection 2:")
    for idx, row in enumerate(block_2, 1):
        parts.append(f"{idx}: {row['text']}")
    parts.append("")
    parts.append("JSON result:")
    return "\n".join(parts)
```

8.4 Token-budgeted block sizing

V1 ports the block-size calculation from Trummer-style block joins:

```
def optimal_block_size(s1, s2, s3, token_threshold, prompt_size, selectivity_estimate):
    estimate = max(float(selectivity_estimate), 0.0000001)
    available = max(1.0, float(token_threshold - prompt_size))
    numerator = math.sqrt(s1 * s1 * s2 * s2 + s1 * s2 * s3 * estimate * available) - s1 * s2
    b1 = math.floor(numerator / (s1 * s3 * estimate))
    b1 = max(1, b1)
    b2 = math.floor(available - b1 * s1)
    b2 = math.floor(b2 / max(1.0, s2 + b1 * s3 * estimate))
    b2 = max(1, b2)
    return b1, b2
```

The actual LLM call uses JSON schema output constrained to movie IDs in the current movie block:

```
response = client.chat(
    messages=[{"role": "user", "content": prompt}],
    model=model,
    max_tokens=max_tokens,
    temperature=0,
    response_schema=join_response_schema(block_1),
)
```

8.5 Simple usage

```
cd "/Users/annremizova/Desktop/lab m2"
python3 common_benchmark_v3/scripts/run_heterogen_v1.py \
  --api-base http://127.0.0.1:11434 \
  --model ollama/qwen3:1.7b \
  --output-dir /tmp/heterogen_v1_example
```

Chapter 9

Trummer Heterogen V2: Exact-ID Row-Wise Cascade

9.1 Idea

V2 changes the candidate unit. Instead of asking the LLM to discover both the join and the semantic match inside large blocks, V2 first creates exact `movie_id = tconst` candidates deterministically. Then each candidate pair is scored by the cheap model. Confident candidates are accepted or rejected. The uncertain candidates are verified by the expensive model in fallback batches.

9.2 Code path

- `project Trummer/heterogen_v2/run_use_case3.py`
- `project Trummer/heterogen_v2/trummer_join/cascade.py`
- Benchmark wrapper: `common_benchmark_v3/scripts/run_heterogen_v2.py`

9.3 Candidate generation

```
def exact_id_candidates(
    movies: Iterable[dict[str, str]],
    reviews: Iterable[dict[str, str]],
) -> list[Candidate]:
    movies_by_id: dict[str, list[dict[str, str]]] = {}
    for movie in movies:
        movies_by_id.setdefault(str(movie.get("movie_id", "")), []).append(movie)
    candidates: list[Candidate] = []
    for review in reviews:
        for movie in movies_by_id.get(str(review.get("tconst", "")), []):
            candidates.append(Candidate(len(candidates) + 1, movie, review))
    return candidates
```

9.4 Cheap pair prompt

```
def _pair_prompt(candidate: Candidate, predicate: str, max_review_chars: int) -> str:
    return (
        "Classify whether this candidate pair satisfies the predicate.\n"
        "Return exactly one token: 1 for yes, 0 for no.\n\n"
        f"Predicate: {predicate}\n"
        f"Movie: {candidate.movie.get('text', '')}\n"
        f"Review: {candidate.review.get('text', '')[:max_review_chars]}\n\n"
        "Answer:"
    )
```

9.5 Expensive fallback batch prompt

```
def _batch_prompt(candidates: list[Candidate], predicate: str, max_review_chars: int) -> str:
    parts = [
        "Evaluate the explicit candidate pairs below.",
        f"Predicate: {predicate}",
        "Each pair has a label such as PAIR_12.",
        "Return only matching PAIR labels separated by commas, for example: PAIR_2, PAIR_7.",
        'Return "none" if no candidate satisfies it. Do not explain.',
        "",
    ]
    for candidate in candidates:
        parts.extend([
            f"PAIR_{candidate.candidate_id}:",
            f"Movie: {candidate.movie.get('text', '')}",
            f"Review: {candidate.review.get('text', '')[:max_review_chars]}",
            "",
        ])
    parts.append("Matching candidate IDs:")
    return "\n".join(parts)
```

9.6 Routing

```
elif _is_confident(score, routing_threshold) and score >= 0:
    metrics.cheap_early_accepts += 1
    accepted.append(candidate)
    decisions.append(_decision(candidate, score, "cheap_accept"))
elif _is_confident(score, routing_threshold):
    metrics.cheap_early_rejects += 1
    decisions.append(_decision(candidate, score, "cheap_reject"))
else:
    metrics.expensive_candidates += 1
    uncertain.append(candidate)
    decisions.append(_decision(candidate, score, "expensive"))
```

9.7 Simple usage

```
cd "/Users/annremizova/Desktop/lab m2"
python3 common_benchmark_v3/scripts/run_heterogen_v2.py \
  --api-base http://127.0.0.1:11434 \
  --cheap-model ollama/qwen3:0.6b \
  --expensive-model ollama/qwen3:1.7b \
  --manual-confidence-threshold 2 \
  --output-dir /tmp/heterogen_v2_example
```

Chapter 10

Trummer Heterogen V2_2: Structured-Pruned Block Join

10.1 Idea

V2_2 keeps the block-join prompt strategy from V1 but moves structured movie conditions out of the LLM workload. A cheap structured parser or conservative regex extractor maps the natural-language question to movie-table filters. Those filters prune movies; reviews are pruned to matching `tconst` values; the LLM receives only the remaining movie/review blocks and evaluates the semantic review predicate.

10.2 Code path

- `project Trummer/heterogen_v2_2/run_use_case3.py`
- `project Trummer/heterogen_v2_2/trummer_join/structured_filter.py`
- `project Trummer/heterogen_v2_2/trummer_join/operators.py`
- Benchmark wrapper: `common_benchmark_v3/scripts/run_heterogen_v2_2.py`
- Multi-question wrapper: `common_benchmark_10q/scripts/run_method.py`, function `run_v2_2()`

10.3 Structured parser prompt

From `project Trummer/heterogen_v2_2/trummer_join/structured_filter.py`:

```
STRUCTURED_PARSER_SYSTEM = textwrap.dedent("""
You are a SUQL semantic parser for an IMDb movie table. Convert the user's
question into one SUQL query.

Table: movies
Columns:
  movie_id TEXT
  title TEXT
  year INTEGER
  runtime INTEGER
  director TEXT
  genres TEXT

There is also conceptual review text available only through:
  answer(review, '<yes/no question>') = 'Yes'

Rules:
1. Put every movie-table condition in normal SQL over movies.
2. Use answer(review, ...) only for conditions that require reading review text.
3. Do not use answer() for director, title, year, runtime, movie_id, or genres.
...
```

```
""").strip()
```

10.4 Pruning flow

```
def prune_movie_frame(frame: pd.DataFrame, question: str, *, api_base: str, parser_model: str
    | None, use_llm: bool = True, suql_query: str | None = None):
    filters = extract_structured_filters(question, frame.columns)
    model = parser_model or ""
    if suql_query or (use_llm and model):
        try:
            query = suql_query or nl_to_structural_suql(...)
            pruned, structural_sql = apply_suql_structural_pruning(frame, query)
            return pruned, StructuredPruningResult(
                mode="suql_sqlite",
                filters=filters,
                suql_query=query,
                structural_sql=structural_sql,
                parser_model=model,
                semantic_predicate=semantic_predicate_from_suql(query, question),
            )
        except Exception as exc:
            pruned = apply_structured_filters(frame, filters).reset_index(drop=True)
            return pruned, StructuredPruningResult(mode="regex_fallback", ...)
```

The benchmark wrapper then uses the pruned movie IDs to prune reviews:

```
movie_ids = set(pruned_movies["movie_id"].astype(str)) if "movie_id" in pruned_movies else set
    ()
pruned_reviews = pd.DataFrame(
    [review for review in reviews if str(review.get("tconst", "")) in movie_ids],
    columns=review_frame.columns,
).reset_index(drop=True)
```

10.5 Simple usage

```
cd "/Users/annremizova/Desktop/lab m2"
python3 common_benchmark_v3/scripts/run_heterogen_v2_2.py \
  --api-base http://127.0.0.1:11434 \
  --structured-parser-model ollama/qwen3:0.6b \
  --model ollama/qwen3:1.7b \
  --output-dir /tmp/heterogen_v2_2_example
```

Chapter 11

Trummer Heterogen V2_3: Batched Exact-ID Cascade

11.1 Idea

V2_3 keeps the exact-ID candidate set from V2, but changes request granularity. Instead of one cheap call per candidate, it scores cheap candidates in batches. Instead of small fallback groups, it coalesces uncertain candidates into larger expensive batches. The semantic meaning is the same as V2; the physical plan is more batch-oriented.

11.2 Code path

- `project Trummer/heterogen_v2_3/run_use_case3.py`
- `project Trummer/heterogen_v2_3/trummer_join/cascade.py`
- Benchmark wrapper: `common_benchmark_v3/scripts/run_heterogen_v2_3.py`
- Multi-question wrapper: `common_benchmark_10q/scripts/run_method.py`, function `run_v2_3()`

11.3 Batched cheap prompt

```
def _cheap_batch_prompt(candidates: list[Candidate], predicate: str, max_review_chars: int) ->
     str:
    parts = [
        "Classify every explicit candidate pair below.",
        f"Predicate: {predicate}",
        "For each candidate return candidate_id and answer 1 for yes or 0 for no.",
        'Return JSON exactly in this shape: {"decisions":[{"candidate_id":1,"answer":0}]}.',
        "Return every candidate exactly once and do not explain.",
        "",
    ]
    for candidate in candidates:
        parts.extend([
            f"CANDIDATE_{candidate.candidate_id}:",
            f"Movie: {candidate.movie.get('text', '')}",
            f"Review: {candidate.review.get('text', '')[:max_review_chars]}",
            "",
        ])
    parts.append("JSON decisions:")
    return "\n".join(parts)
```

The request uses Ollama structured output when available:

```
request_payload = {
    "model": _plain_model(self.config.cheap_model),
    "messages": [{"role": "user", "content": prompt}],
```

```
    "stream": False,
    "think": False,
    "format": {
        "type": "object",
        "properties": {
            "decisions": {
                "type": "array",
                "items": {
                    "type": "object",
                    "properties": {
                        "candidate_id": {"type": "integer", "enum": ids},
                        "answer": {"type": "integer", "enum": [0, 1]},
                    },
                    "required": ["candidate_id", "answer"],
                },
            }
        },
        "required": ["decisions"],
    },
    "options": {"temperature": 0, "num_predict": max(64, len(candidates) * 12)},
}
```

11.4 Batched expensive prompt

```
def _expensive_batch_prompt(candidates: list[Candidate], predicate: str, max_review_chars: int
    ) -> str:
    parts = [
        "Evaluate the explicit candidate pairs below.",
        f"Predicate: {predicate}",
        "Return only the matching candidate IDs.",
        "Do not explain.",
        "",
    ]
    for candidate in candidates:
        parts.extend([
            f"PAIR_{candidate.candidate_id}:",
            f"Movie: {candidate.movie.get('text', '')}",
            f"Review: {candidate.review.get('text', '')[:max_review_chars]}",
            "",
        ])
    parts.append("Matching candidate IDs:")
    return "\n".join(parts)
```

11.5 Simple usage

```
cd "/Users/annremizova/Desktop/lab m2"
python3 common_benchmark_v3/scripts/run_heterogen_v2_3.py \
  --api-base http://127.0.0.1:11434 \
  --cheap-model ollama/qwen3:0.6b \
  --expensive-model ollama/qwen3:1.7b \
  --cheap-batch-size 8 \
  --expensive-batch-size 32 \
  --output-dir /tmp/heterogen_v2_3_example
```

Chapter 12

Trummer Heterogen V3: Structured-Pruned Row-Wise Cascade

12.1 Idea

V3 combines V2_2 and V2. It first applies structured pruning, then generates exact-ID candidates only inside the pruned movie/review subset. It scores those pruned candidates row by row with the cheap model and sends uncertain candidates to the expensive model. This reduces the candidate set before the cascade.

12.2 Code path

- `project Trummer/heterogen_v3/run_use_case3.py`
- `project Trummer/heterogen_v3/trummer_join/structured_filter.py`
- `project Trummer/heterogen_v3/trummer_join/cascade.py`
- Benchmark wrapper: `common_benchmark_v3/scripts/run_heterogen_v3.py`
- Multi-question wrapper: `common_benchmark_10q/scripts/run_method.py`, function `run_v3()`

12.3 Runner flow

From `project Trummer/heterogen_v3/run_use_case3.py`:

```
movies, reviews, structured_filters = prune_inputs(
    input_movies,
    input_reviews,
    args.question,
    args.year,
    api_base=args.api_base,
    parser_model=args.structured_parser_model or args.cheap_model,
    request_timeout=args.request_timeout,
    use_llm=not args.dry_run and not args.disable_llm_structured_parser,
)
predicate = args.predicate or structured_filters.semantic_predicate or
    semantic_predicate_from_question(args.question)

config = CascadeConfig(
    api_base=args.api_base,
    cheap_model=args.cheap_model,
    expensive_model=args.expensive_model,
    cascade_target=args.cascade_target,
    calibration_budget=args.calibration_budget,
    expensive_batch_size=args.expensive_batch_size,
    max_expensive_calls=args.max_expensive_calls,
    request_timeout=args.request_timeout,
```

```
)

rows, decisions, metrics = join.run(movies, reviews, predicate)
```

12.4 Review pruning by selected movie IDs

```
movie_ids = set(pruned_movies["movie_id"].astype(str)) if "movie_id" in pruned_movies else set
    ()
pruned_reviews = pd.DataFrame(
    [review for review in reviews if str(review.get("tconst", "")) in movie_ids],
    columns=review_frame.columns,
).reset_index(drop=True)
```

12.5 Expensive-call cap

V3 chooses a fallback batch size large enough to respect `-max-expensive-calls`:

```
expensive_batch_size = max(
    self.config.expensive_batch_size,
    _minimum_batch_size(uncertain, self.config.max_expensive_calls),
)
fallback_batches = list(_blocks(uncertain, expensive_batch_size))
```

12.6 Simple usage

```
cd "/Users/annremizova/Desktop/lab m2"
python3 common_benchmark_v3/scripts/run_heterogen_v3.py \
  --api-base http://127.0.0.1:11434 \
  --structured-parser-model ollama/qwen3:0.6b \
  --cheap-model ollama/qwen3:0.6b \
  --expensive-model ollama/qwen3:1.7b \
  --max-expensive-calls 4 \
  --output-dir /tmp/heterogen_v3_example
```

Chapter 13

Trummer Heterogen V3_2: Structured-Pruned Batched Cascade

13.1 Idea

V3_2 combines V3 pruning with V2_3 batching. It applies SUQL-style structured pruning first, prunes reviews by selected movie IDs, generates exact-ID candidates, scores them in cheap batches, and sends uncertain candidates to larger expensive batches. This is the most complete Heterogen implementation in the current larger benchmarks because it combines both major optimizations:

- fewer candidates through deterministic pruning;
- fewer LLM requests through batched cascade calls.

13.2 Code path

- `project Trummer/heterogen_v3_2/run_use_case3.py`
- `project Trummer/heterogen_v3_2/trummer_join/structured_filter.py`
- `project Trummer/heterogen_v3_2/trummer_join/cascade.py`
- Ten-question wrapper: `common_benchmark_10q/scripts/run_method.py`, function `run_v3_2()`

13.3 Runner flow

From `project Trummer/heterogen_v3_2/run_use_case3.py`:

```
movies, reviews, structured_filters = prune_inputs(
    input_movies,
    input_reviews,
    args.question,
    args.year,
    api_base=args.api_base,
    parser_model=args.structured_parser_model or args.cheap_model,
    request_timeout=args.request_timeout,
    use_llm=not args.dry_run and not args.disable_llm_structured_parser,
)
predicate = args.predicate or structured_filters.semantic_predicate or
    semantic_predicate_from_question(args.question)

config = CascadeConfig(
    api_base=args.api_base,
    cheap_model=args.cheap_model,
    expensive_model=args.expensive_model,
    cascade_target=args.cascade_target,
    calibration_budget=args.calibration_budget,
```

```
    manual_confidence_threshold=args.manual_confidence_threshold,
    cheap_batch_size=args.cheap_batch_size,
    expensive_batch_size=args.expensive_batch_size,
    request_timeout=args.request_timeout,
)
rows, decisions, metrics = join.run(movies, reviews, predicate)
```

13.4 Ten-question wrapper

From `common_benchmark_10q/scripts/run_method.py`:

```
def run_v3_2(args: argparse.Namespace, spec: dict, output_dir: Path) -> dict:
    v3_2_root = LAB_ROOT / "project Trummer" / "heterogen_v3_2"
    sys.path.insert(0, str(v3_2_root))
    from trummer_join.cascade import CascadeConfig, CascadeJoin, metrics_dict
    from trummer_join.structured_filter import prune_movie_frame

    input_movies = load_movies(args.question_dir)
    input_reviews = load_reviews(args.question_dir)
    movies_frame, pruning = prune_movie_frame(...)
    movie_ids = set(movies_frame["movie_id"].astype(str))
    reviews_frame = pd.DataFrame(
        [row for row in input_reviews if str(row.get("tconst", "")) in movie_ids],
        columns=input_reviews_frame.columns,
    ).reset_index(drop=True)

    rows, decisions, metrics = CascadeJoin(config).run(
        movies,
        reviews,
        spec["semantic_question"],
    )
```

The metrics record where each condition was handled:

```
"structured_condition_location": "deterministic_prefilter",
"join_condition_location": "deterministic_prefilter",
"semantic_condition_location": "batch_cascade",
```

13.5 Simple usage

```
cd "/Users/annremizova/Desktop/lab m2"
python3 common_benchmark_10q/scripts/run_method.py \
  --method v3_2 \
  --question-dir question_01_year_2001_negative \
  --api-base http://127.0.0.1:11434 \
  --cheap-model ollama/gemma4:e2b \
  --expensive-model ollama/gemma4:e4b \
  --output-dir /tmp/heterogen_v3_2_example
```

Chapter 14

Prompt Examples From The Code

14.1 SUQL parser prompt example

```
You are a SUQL (Structured and Unstructured Query Language) semantic parser.
SUQL extends SQL with two free-text functions:
  - answer(free_text_column, 'question')  -> 'Yes' | 'No' | short string
  - summary(free_text_column)             -> a short prose summary

Rules:
1. Use plain SQL WHERE clauses for ALL structured predicates.
2. Use answer() ONLY for predicates that require understanding the review text.
3. Always apply structured filters BEFORE answer() in the WHERE clause.
...
```

14.2 SUQL answer prompt example

```
Review:
<first 1500 characters of one movie review>

Question: Does the reviewer find this movie terrifying or scary?
```

System:

```
You are evaluating a single movie review to answer a question about it.

Rules:
- If the question is a yes/no question, answer ONLY with 'Yes' or 'No' (no other text).
- If the question asks for a specific piece of information, answer with a brief string.
- Do not explain your reasoning. Output only the answer.
```

14.3 Stage 1 and Stage 2 binary scorer prompt

```
Classify whether the review answers the question with Yes.
Return exactly one token: 1 for Yes, 0 for No.

Question: <semantic review question>
Review: <first 1800 characters of one review>

Answer:
```

14.4 Heterogen V1 and V2_2 block prompt

```
Find every movie in Collection 1 that has a review in Collection 2 satisfying this predicate:
<predicate>
Collection 1 contains structured movie rows. Collection 2 contains review chunks.
A review belongs to a movie only when tconst is exactly equal to movie_id.
```

```
Return JSON only: {"matching_movie_ids":["tt123"]}.
If there are no matches, return {"matching_movie_ids":[]}.
Copy each returned movie_id exactly from Collection 1.
Do not add prose or a completion marker.

Collection 1:
1: movie_id=...; title=...; year=...; director=...; runtime=...; genres=...

Collection 2:
1: tconst=...; review=...

JSON result:
```

14.5 Heterogen V2 cheap pair prompt

```
Classify whether this candidate pair satisfies the predicate.
Return exactly one token: 1 for yes, 0 for no.

Predicate: <semantic predicate>
Movie: movie_id=...; title=...; year=...; director=...; runtime=...; genres=...
Review: tconst=...; review=...

Answer:
```

14.6 Heterogen V2_3 and V3_2 cheap batch prompt

```
Classify every explicit candidate pair below.
Predicate: <semantic predicate>
For each candidate return candidate_id and answer 1 for yes or 0 for no.
Return JSON exactly in this shape: {"decisions":[{"candidate_id":1,"answer":0}]}.
Return every candidate exactly once and do not explain.

CANDIDATE_1:
Movie: movie_id=...; title=...; year=...; director=...; runtime=...; genres=...
Review: tconst=...; review=...

JSON decisions:
```

14.7 Heterogen V2/V3 expensive fallback prompt

```
Evaluate the explicit candidate pairs below.
Predicate: <semantic predicate>
Each pair has a label such as PAIR_12.
Return only matching PAIR labels separated by commas, for example: PAIR_2, PAIR_7.
Return "none" if no candidate satisfies it. Do not explain.

PAIR_1:
Movie: movie_id=...; title=...
Review: tconst=...; review=...

Matching candidate IDs:
```

Chapter 15

Common Benchmark Runners

15.1 One-question all-method runner

`common_benchmark_v3/scripts/run_all.py` compares SUQL baseline plus selected Heterogen variants. It builds commands for each method and then evaluates the output directory:

```
if not args.skip_suql:
    commands.append([
        args.python, str(ROOT / "scripts" / "run_suql_baseline.py"), *common,
        "--model", args.expensive_model,
        "--output-dir", str(experiment / "suql_baseline"),
    ])
...
run([args.python, str(ROOT / "scripts" / "evaluate_and_plot.py"), "--outputs-dir", str(
    experiment)], env)
```

Simple run:

```
cd "/Users/annremizova/Desktop/lab m2"
python3 common_benchmark_v3/scripts/run_all.py \
  --api-base http://127.0.0.1:11434 \
  --cheap-model ollama/qwen3:0.6b \
  --expensive-model ollama/qwen3:1.7b \
  --repetitions 3
```

15.2 Heterogen-only runner

`common_benchmark_v3/scripts/run_all_heterogen.py` runs V1, V2, V2_2, V2_3, and V3 without SUQL:

```
cd "/Users/annremizova/Desktop/lab m2"
python3 common_benchmark_v3/scripts/run_all_heterogen.py \
  --api-base http://127.0.0.1:11434 \
  --cheap-model ollama/qwen3:0.6b \
  --expensive-model ollama/qwen3:1.7b \
  --repetitions 3
```

15.3 Ten-question runner

`common_benchmark_10q/scripts/run_all.py` runs methods across all ten question directories. The default methods are SUQL baseline, V2_3, V3, and V3_2:

```
parser.add_argument(
    "--methods",
    nargs="+",
    choices=sorted(METHOD_DIRS),
    default=["suql", "v2_3", "v3", "v3_2"],
)
```

Simple run:

```
cd "/Users/annremizova/Desktop/lab m2"
python3 common_benchmark_10q/scripts/build_datasets.py
python3 common_benchmark_10q/scripts/run_all.py \
  --api-base http://127.0.0.1:11434 \
  --cheap-model ollama/gemma4:e2b \
  --expensive-model ollama/gemma4:e4b \
  --repetitions 3 \
  --output-dir outputs/local_small_example
```

15.4 Threshold sweep

`common_benchmark_thresholds/scripts/run_threshold_sweep.py` sweeps the manual confidence threshold used by V2 and V2_3:

```
cd "/Users/annremizova/Desktop/lab m2"
python3 common_benchmark_thresholds/scripts/run_threshold_sweep.py \
  --api-base http://127.0.0.1:11434 \
  --cheap-model ollama/qwen3:0.6b \
  --expensive-model ollama/qwen3:1.7b \
  --thresholds 0,1,2,3 \
  --repetitions 3
```

Chapter 16

Output Artifacts

Most method runners write the same artifact shape:

File	Meaning
`run_metrics.json`	Method config, timing, call counts, final row count, found movie IDs, pruning metadata.
`found_rows.csv` or `final_movies.csv`	Deduplicated final movie-level answers.
`joined_evidence.csv`	Movie-review pairs or candidate pairs accepted by the method.
`cascade_decisions.csv`	Candidate-level cheap/expensive routing decisions for cascade methods.
`join_stats.csv`	Block-level token/time/parser stats for block-join methods.
`run_metrics_repetitions.csv`	Per-repetition metrics before averaging.
`comparison.csv`, `aggregate.csv`, `all_metrics.csv`	Evaluation tables produced by benchmark-level scripts.
`summary.md` and plots	Human-readable aggregate summaries and quality/time/call plots.

Chapter 17

Practical Interpretation Notes

- Compare methods by quality, time, and call mix together. A method with fewer calls can still be slower if its fallback batches are large.
- For cascade methods, inspect `cheap_calls`, `expensive_calls`, `cheap_early_accepts`, `cheap_early_rejects`, and `expensive_candidates`.
- For structured-pruned methods, inspect `original_movies`, `pruned_movies`, `original_reviews`, `pruned_reviews`, and `structured_pruning`.
- For block joins, inspect `join_stats.csv` because token overflow and prompt block size directly affect recall and runtime.
- Dry runs validate data wiring and evaluation code only. They are not LLM quality results.